

Quickplay AI Studio Public API - Developer Guide

API Version: 6.3 **Base URL:** <https://verticalizer.videoai.wbd.com/>

Authentication

All endpoints require an API key passed in the **x-api-key** header.

```
x-api-key: YOUR_API_KEY
```

Endpoints

POST /v2/upload

Add or update a media asset with optional metadata. If an asset with the same **ext_id** already exists, it will be updated and the existing **media_id** will be returned.

Request Body

Field	Type	Required	Description
master	MediaAsset	Yes	Primary media asset
proxy	MediaAsset	No	Proxy (lower-resolution) media asset
priority	integer	No	Processing priority, 0-1000 (default 0). 0 is lowest, 1000 is highest. Jobs are processed based on this priority and request time -- earlier requests win within the same priority.
email	string	No	Email address of the requesting user.
ai_flags	string[]	No	AI processing flags. When provided, the request is processed asynchronously and returns HTTP 202 with a job_id . See AI flags and async response sections below.
metadata	array	No	Localized metadata entries

AI flags

Supported values for **ai_flags**:

Value	Description
AUTO_GENERATE_ALL	Run all auto-generation steps

Value	Description
<code>AUTO_GENERATE_DEFAULT</code>	Run the default auto-generation steps
<code>AUTO_GENERATE_SOURCE_METADATA</code>	Auto-generate source metadata
<code>AUTO_DETECT_SCENES</code>	Auto-detect scenes in the asset

The array must match **exactly one** of the following combinations:

1. `[AUTO_GENERATE_ALL]` by itself
2. `[AUTO_GENERATE_DEFAULT]` by itself
3. Any non-empty subset of `{AUTO_GENERATE_SOURCE_METADATA, AUTO_DETECT_SCENES}` (either one, or both)

`AUTO_GENERATE_ALL` and `AUTO_GENERATE_DEFAULT` are mutually exclusive with each other **and** with the granular flags. For example, `[AUTO_GENERATE_ALL, AUTO_DETECT_SCENES]` and `[AUTO_GENERATE_DEFAULT, AUTO_GENERATE_SOURCE_METADATA]` are both invalid.

Valid examples:

```
[ "AUTO_GENERATE_ALL" ]
[ "AUTO_GENERATE_DEFAULT" ]
[ "AUTO_GENERATE_SOURCE_METADATA" ]
[ "AUTO_DETECT_SCENES" ]
[ "AUTO_GENERATE_SOURCE_METADATA", "AUTO_DETECT_SCENES" ]
```

Invalid examples:

```
[ "AUTO_GENERATE_ALL", "AUTO_GENERATE_DEFAULT" ]
[ "AUTO_GENERATE_ALL", "AUTO_DETECT_SCENES" ]
[ "AUTO_GENERATE_DEFAULT", "AUTO_GENERATE_SOURCE_METADATA" ]
```

MediaAsset fields:

Field	Type	Required	Description
<code>ext_id</code>	string	Yes	Identifier for the asset in the external MAM/DAM system
<code>uri</code>	string	Yes	Location of the asset. Supported schemes: <code>https</code> , <code>s3</code> , <code>gcs</code>

Each MediaAsset must include **exactly one** of:

- `media_profile` (array) -- inline profile defining the media file structure
- `media_profile_name` (string) -- name of a pre-configured media profile

These two fields are mutually exclusive.

Metadata entry fields:

Field	Type	Description
<code>language</code>	string	Language code (e.g. <code>en_CA</code>)
<code>title</code>	string	Asset title
<code>description</code>	string	Asset description

Media Profile

When providing an inline `media_profile`, each entry is a track with a `group_type` discriminator:

Video track (`group_type: video`)

Field	Type	Description
<code>track_info.codec</code>	string	AVC, HEVC, MPEG-4 Part 2, Apple ProRes 422, AVC-Intra, JPEG2000
<code>track_info.bitRate</code>	integer	Bit rate in bits per second
<code>track_info.frameRate</code>	object	{ <code>numerator</code> , <code>denominator</code> } (e.g. 30000/1001 for 29.97fps)
<code>track_info.frameSize</code>	object	{ <code>width</code> , <code>height</code> } in pixels

Audio track (`group_type: audio`)

Field	Type	Description
<code>audio_kind</code>	string	Audio content type: PRM, SAP, HI, DV, DX, MX, FX, FFX, ME, OP, MESP, DME, NDME, PNAR, ONAR, VO, VI, CM, LCM, MOS
<code>audio_group</code>	string	Channel configuration: ST, DM, LtRt, DNS, 3.0, 4.0, 5.0, 5.1, 5.1EX, 6.0, 6.1, 7.0DS, 7.1DS, 7.1SDS, HA, VA
<code>language</code>	string	Language code (e.g. <code>en-US</code>)
<code>track_info.codec</code>	string	AAC, AC3, EAC3, FLAC, PCM-16, PCM-24
<code>track_info.bitRate</code>	integer	Bit rate in bits per second
<code>track_info.sampleRate</code>	integer	Sample rate in Hz
<code>channel_layouts</code>	array	Channel mapping entries with <code>channel_locator</code> (required) and <code>audio_channel</code>

Supported `audio_channel` values: L, R, C, LFE, Ls, Rs, Lss, Rss, Lrs, Rrs, Lc, Rc, Cs, HI, VIN, Lt, Rt, Lst, Rst, S, M1, M2

Closed caption track (group_type: cc)

Field	Type	Description
format	string	CEA-608, EIA-608, CEA-708, EIA-708
language	string	Language code (e.g. en-US)

Example: Upload with inline media profile

```
{
  "master": {
    "ext_id": "asset-001",
    "uri": "s3://bucket/media/video.mxf",
    "media_profile": [
      {
        "group_type": "video",
        "track_info": {
          "codec": "AVC",
          "bitRate": 50000000,
          "frameRate": { "numerator": 30000, "denominator": 1001 },
          "frameSize": { "width": 1920, "height": 1080 }
        }
      },
      {
        "group_type": "audio",
        "audio_kind": "PRM",
        "audio_group": "ST",
        "language": "en-US",
        "track_info": {
          "codec": "PCM-24",
          "bitRate": 2304000,
          "sampleRate": 48000
        },
        "channel_layouts": [
          { "channel_locator": "1", "audio_channel": "L" },
          { "channel_locator": "2", "audio_channel": "R" }
        ]
      },
      {
        "group_type": "cc",
        "format": "CEA-608",
        "language": "en-US"
      }
    ]
  },
  "proxy": {
    "ext_id": "asset-001-proxy",
    "uri": "s3://bucket/media/video_proxy.mp4",
  }
}
```

```

"media_profile": [
  {
    "group_type": "video",
    "track_info": {
      "codec": "AVC",
      "bitRate": 5000000,
      "frameRate": { "numerator": 30000, "denominator": 1001 },
      "frameSize": { "width": 1280, "height": 720 }
    }
  },
  {
    "group_type": "audio",
    "audio_kind": "PRM",
    "audio_group": "ST",
    "language": "en-US",
    "track_info": {
      "codec": "AAC",
      "bitRate": 192000,
      "sampleRate": 48000
    },
    "channel_layouts": [
      { "channel_locator": "1", "audio_channel": "L" },
      { "channel_locator": "2", "audio_channel": "R" }
    ]
  }
]
},
"metadata": [
  {
    "language": "en-US",
    "title": "Sample Video",
    "description": "A sample media asset"
  }
]
}

```

Example: Upload with pre-configured profile name

```

{
  "master": {
    "ext_id": "asset-002",
    "uri": "s3://bucket/media/video.mxf",
    "media_profile_name": "HD-1080p-Stereo"
  },
  "proxy": {
    "ext_id": "asset-002-proxy",
    "uri": "s3://bucket/media/video_proxy.mp4",
    "media_profile_name": "Proxy-720p"
  }
}

```

```
},
"metadata": [
  {
    "language": "en-US",
    "title": "Sample Video",
    "description": "A sample media asset"
  }
]
}
```

Example: Async upload with ai_flags

Including `ai_flags` switches the endpoint to asynchronous mode. The API returns **202 Accepted** with a `job_id` that can be polled via `/v2/status`.

```
{
  "master": {
    "ext_id": "asset-003",
    "uri": "s3://bucket/media/video.mxf",
    "media_profile_name": "HD-1080p-Stereo"
  },
  "proxy": {
    "ext_id": "asset-003-proxy",
    "uri": "s3://bucket/media/video_proxy.mp4",
    "media_profile_name": "Proxy-720p"
  },
  "priority": 500,
  "email": "producer@example.com",
  "ai_flags": ["AUTO_GENERATE_SOURCE_METADATA", "AUTO_DETECT_SCENES"],
  "metadata": [
    {
      "language": "en-US",
      "title": "Sample Video",
      "description": "A sample media asset"
    }
  ]
}
```

Response (202):

```
{
  "job_id": "1015825c-7c90-4042-8513-24585fb9c235",
  "status": "Queued"
}
```

Responses

200 OK -- Asset already exists; returns existing asset info.

201 Created -- Asset created successfully.

Response body for both 200 and 201:

```
{
  "job_id": "string",
  "ext_id": "string",
  "uri": "string",
  "media_id": "string",
  "duration": 0,
  "mimeType": "string",
  "status": "string"
}
```

Field	Type	Description
<code>job_id</code>	string	Job ID for tracking upload processing via <code>/v2/status</code>
<code>ext_id</code>	string	External asset identifier
<code>uri</code>	string	Location of the uploaded asset
<code>media_id</code>	string	Internal identifier assigned to the asset
<code>duration</code>	integer	Duration in milliseconds
<code>mimeType</code>	string	MIME type of the asset
<code>status</code>	string	Processing status

Async response with `ai_flags`

When `ai_flags` is provided in the request, the upload is processed **asynchronously** and the API always returns **202 Accepted** instead of 200/201. The response contains only the `job_id` and a `status` of `Queued`. Poll `/v2/status` with the returned `job_id` to track progress.

```
{
  "job_id": "1015825c-7c90-4042-8513-24585fb9c235",
  "status": "Queued"
}
```

POST /v3/upload

Same purpose as [/v2/upload](#), but **master** and **proxy** are **objects** that wrap a **media_assets** array, so a single request can register multiple master and proxy assets together (e.g. multi-track or multi-rendition deliveries). The other request fields (**priority**, **email**, **ai_flags**, **metadata**) and response codes are identical to [/v2/upload](#).

Request Body

Field	Type	Required	Description
master	MediaAssetV3Group	Yes	Master asset group containing a media_assets array.
proxy	MediaAssetV3Group	No	Proxy asset group containing a media_assets array.
priority	integer	No	Processing priority, 0-1000 (default 0). Same semantics as /v2/upload .
email	string	No	Email address of the requesting user.
ai_flags	string[]	No	AI processing flags. Same combination rules as /v2/upload . When provided, the request is processed asynchronously and returns 202.
metadata	array	No	Localized metadata entries.

MediaAssetV3Group fields:

Field	Type	Required	Description
ext_id	string	Yes	Identifier for this asset group in the external MAM / DAM system.
media_assets	MediaAssetV3Item[]	Yes	One or more media assets in the group. At least one entry.

MediaAssetV3Item fields:

Field	Type	Required	Description
ext_asset_id	string	Yes	Identifier for the asset within this upload group
uri	string	Yes	Location of the asset. Supported schemes: https , s3 , gcs
media_profile	MediaProfileTrack[]	Yes	Inline media profile (video/audio/cc tracks). See /v2/upload media profile for track structure.

Note: unlike [/v2/upload](#), **MediaAssetV3Item** does not support **media_profile_name** -- only inline **media_profile** is accepted.

Example: Master only (no proxy)

```
{
  "master": {
    "ext_id": "asset-group-003",
    "media_assets": [
      {
        "ext_asset_id": "master-video-003",
        "uri": "s3://bucket/media/video.mxf",
        "media_profile": [
          {
            "group_type": "video",
            "track_info": {
              "codec": "AVC",
              "bitRate": 50000000,
              "frameRate": { "numerator": 30000, "denominator": 1001 },
              "frameSize": { "width": 1920, "height": 1080 }
            }
          }
        ]
      }
    ]
  },
  "priority": 500,
  "email": "producer@example.com",
  "metadata": [
    {
      "language": "en-US",
      "title": "Sample Video",
      "description": "A master-only upload"
    }
  ]
}
```

Example: Multiple master assets with corresponding proxies

```
{
  "master": {
    "ext_id": "asset-group-001",
    "media_assets": [
      {
        "ext_asset_id": "master-video-001",
        "uri": "s3://bucket/media/video.mxf",
        "media_profile": [
          {
            "group_type": "video",
```

```

        "track_info": {
            "codec": "AVC",
            "bitRate": 50000000,
            "frameRate": { "numerator": 30000, "denominator": 1001 },
            "frameSize": { "width": 1920, "height": 1080 }
        }
    }
}
],
{
    "ext_asset_id": "master-audio-001",
    "uri": "s3://bucket/media/audio.wav",
    "media_profile": [
        {
            "group_type": "audio",
            "audio_kind": "PRM",
            "audio_group": "ST",
            "language": "en-US",
            "track_info": {
                "codec": "PCM-24",
                "bitRate": 2304000,
                "sampleRate": 48000
            },
            "channel_layouts": [
                { "channel_locator": "1", "audio_channel": "L" },
                { "channel_locator": "2", "audio_channel": "R" }
            ]
        }
    ]
}
]
},
"proxy": {
    "ext_id": "asset-group-001-proxy",
    "media_assets": [
        {
            "ext_asset_id": "proxy-video-001",
            "uri": "s3://bucket/media/video_proxy.mp4",
            "media_profile": [
                {
                    "group_type": "video",
                    "track_info": {
                        "codec": "AVC",
                        "bitRate": 5000000,
                        "frameRate": { "numerator": 30000, "denominator": 1001 },
                        "frameSize": { "width": 1280, "height": 720 }
                    }
                }
            ]
        }
    ]
}
}

```

```
    ]
  },
  "priority": 500,
  "email": "producer@example.com",
  "metadata": [
    {
      "language": "en-US",
      "title": "Sample Video",
      "description": "A sample multi-asset upload"
    }
  ]
}
```

Responses

Same as `/v2/upload`:

- **200 OK** -- asset group already exists
- **201 Created** -- asset group created
- **202 Accepted** -- returned when `ai_flags` is provided; body is `{ "job_id": "...", "status": "Queued" }`
- **400 / 401 / 403** -- standard error responses

POST `/v2/smart-crop`

Generate a video from a sequence of media assets and export to a destination based on a predefined export template. All media assets in the sequence must have been previously added via `/v2/upload`.

Request Body

Field	Type	Required	Description
<code>ext_job_id</code>	string	Yes	External job identifier
<code>sequence</code>	array	Yes	Ordered list of media asset clips
<code>export_template_name</code>	string[]	No	Names of the export templates to apply
<code>priority</code>	integer	No	Processing priority, <code>0-1000</code> (default <code>0</code>). <code>0</code> is lowest, <code>1000</code> is highest. Jobs are processed based on this priority and request time -- earlier requests win within the same priority.
<code>email</code>	string	No	Email address of the requesting user.

Sequence entry fields:

Field	Type	Required	Description
<code>ext_id</code>	string	Yes	External asset identifier. Must be the <code>ext_id</code> of the master asset (not the proxy) previously registered via <code>/v2/upload</code> .
<code>start_timecode</code>	string	No	Start timecode for the clip
<code>end_timecode</code>	string	No	End timecode for the clip

Example

```
{
  "ext_job_id": "my-crop-job-001",
  "sequence": [
    {
      "ext_id": "asset-001",
      "start_timecode": "00:00:10:00",
      "end_timecode": "00:01:30:00"
    },
    {
      "ext_id": "asset-002"
    }
  ],
  "export_template_name": ["HD-1080p-Stereo", "Proxy-720p"]
}
```

Responses

202 Accepted -- Job accepted.

```
{
  "job_id": "1015825c-7c90-4042-8513-24585fb9c235"
}
```

POST /v2/smart-crop/{ext_job_id}/publish

Publish (render and export) the result of an existing smart-crop job using one or more export templates. The video is rendered once per template in `export_template_name`.

The smart-crop job referenced by `ext_job_id` must have been created via `/v2/smart-crop`.

Path Parameters

Name	Type	Required	Description
<code>ext_job_id</code>	string	Yes	External Job ID of the smart-crop job to publish. Must match the <code>ext_job_id</code> used when the job was created via <code>/v2/smart-crop</code> .

Request Body

Field	Type	Required	Description
<code>export_template_name</code>	string[]	Yes	Names of the export templates to render and export. Must contain at least one entry.
<code>priority</code>	integer	No	Processing priority, <code>0-1000</code> (default <code>0</code>). Same semantics as <code>/v2/smart-crop</code> .
<code>email</code>	string	No	Email address of the requesting user.

Example: Publish using multiple export templates

```
POST /v2/smart-crop/my-crop-job-001/publish
```

```
{
  "export_template_name": ["HD-1080p-Stereo", "Proxy-720p"],
  "priority": 0,
  "email": "producer@example.com"
}
```

Responses

202 Accepted -- Publish job accepted.

```
{
  "job_id": "1015825c-7c90-4042-8513-24585fb9c235"
}
```

404 Not Found -- No smart-crop job with the given `ext_job_id`.

400 / 401 / 403 -- Standard error responses.

POST `/v2/status`

Get the status of one or more jobs. Query by **either** AI Studio job IDs or external job IDs (not both).

Request Body

Provide **exactly one** of:

Field	Type	Description
job_id	string[]	AI Studio job IDs
ext_job_id	string[]	External job IDs

Example: Query by job ID

```
{
  "job_id": [
    "1015825c-7c90-4042-8513-24585fb9c235",
    "2a3b4c5d-1234-5678-abcd-ef0123456789"
  ]
}
```

Example: Query by external job ID

```
{
  "ext_job_id": ["ext-job-001", "ext-job-002"]
}
```

Response

Returns an array of job status objects. The shape varies by status:

Common fields (all statuses):

Field	Type	Description
job_id	string	AI Studio job ID
ext_job_id	string	External job ID
status	string	Queued, Processing, Completed, or Failed
progress_percent	integer	Completion percentage (0-100)

Additional fields when status: Completed:

Field	Type	Description
<code>media_assets</code>	array	Output assets produced by the job
<code>media_assets[].uri</code>	string	Location of the output asset
<code>media_assets[].asset_type</code>	string	Type of the output asset (e.g. <code>video</code> , <code>proxy</code>)

Additional fields when `status: Failed`:

Field	Type	Description
<code>error.code</code>	integer	Error code
<code>error.message</code>	string	Error message

Response examples**Queued:**

```
[
  {
    "job_id": "1015825c-7c90-4042-8513-24585fb9c235",
    "ext_job_id": "ext-job-001",
    "status": "Queued",
    "progress_percent": 0
  }
]
```

Completed:

```
[
  {
    "job_id": "3c4d5e6f-2345-6789-bcde-f01234567890",
    "ext_job_id": "ext-job-003",
    "status": "Completed",
    "progress_percent": 100,
    "media_assets": [
      { "uri": "s3://bucket/output/file.mp4", "asset_type": "video" },
      { "uri": "s3://bucket/output/file_proxy.mp4", "asset_type": "proxy" }
    ]
  }
]
```

Failed:

```
[
  {
    "job_id": "4d5e6f7a-3456-789a-cdef-012345678901",
    "ext_job_id": "ext-job-004",
    "status": "Failed",
    "progress_percent": 30,
    "error": {
      "code": 400342,
      "message": "Invalid Video"
    }
  }
]
```

Typical Workflow

1. **Upload assets** -- `POST /v2/upload` for each media asset (master and optional proxy)
2. **Create a smart-crop job** -- `POST /v2/smart-crop` with the sequence of uploaded assets
3. **Poll for status** -- `POST /v2/status` with the `job_id` from step 2 until status is `Completed` or `Failed`

Error Responses

All endpoints return the same error format for standard HTTP errors:

400 Bad Request (invalid or expired API key):

```
{
  "error": {
    "code": 400,
    "message": "API key not valid. Please pass a valid API key.",
    "status": "INVALID_ARGUMENT",
    "details": [
      {
        "@type": "type.googleapis.com/google.rpc.ErrorInfo",
        "reason": "API_KEY_INVALID",
        "domain": "googleapis.com"
      }
    ]
  }
}
```

401 Unauthorized (missing API key):


```
{  
  "message": "UNAUTHENTICATED: Method doesn't allow unregistered callers",  
  "code": 401  
}
```

403 Forbidden (permission denied):

```
{  
  "message": "PERMISSION_DENIED: The API targeted by this request is invalid  
for the given API key.",  
  "code": 403  
}
```